

Everything About GSAP: A Comprehensive Study Guide

Learn to create powerful web animations with GreenSock Animation Platform.

Introduction to GSAP Series

- **Main Goal:** Explore the **GreenSock Animation Platform (GSAP)** for performing powerful animations.
- **Key Topics:**
 - GSAP Basics
 - ScrollTrigger 
 - Animating SVGs 
- **Series Structure:**
 1. GSAP Basics
 2. ScrollTrigger
 3. Animating SVGs
 4. Followed by 5 different projects 

What are Animations?

Animations on web pages refer to any **movement, transition, or visual effect** that:

- Increases user engagement.
- Enhances user experience.
- Encourages users to spend more time on the website.

Example: A simple *hover effect* on a button is a form of animation. 

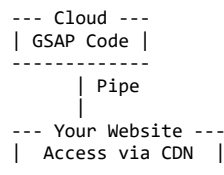
What Exactly is GSAP?

- **Full Form:** **GreenSock Animation Platform.**
- **Type:** A *JavaScript animation library*.
- **Purpose:** Helps create **complex animations** with fewer lines of code and more features.
- It is designed for making *moving animations*, transitions, and various movement-related effects. 

Understanding Libraries

Libraries provide a specific type of code or feature to enhance your website.

- **How they work:**



Similar to connecting a tap to a water tank on a rooftop – you don't bring the tank down, you just open the tap!

- **Examples of Libraries:**
 - **Locomotive JS:** For *smooth scrolling* .
 - **React:** For building entire frontends/UIs .
 - **GSAP:** For handling *moving animations* and effects.
 - **ScrollTrigger:** A *GSAP plugin* for scroll-based animations.
 - **Swiper JS:** For *sliding animations* .

Setting Up GSAP

1. **Create a Project Folder:** Name it `GSAP` on your desktop.
2. **Open in VS Code:** Your code editor will be VS Code.
3. **Create Files:**
 - o `index.html`
 - o `style.css`
 - o `script.js`

4. HTML Boilerplate:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Your HTML elements -->
  <script src="script.js"></script>
</body>
</html>
```

5. Link CSS & JS:

- o CSS: `` in ``.
- o JS: `` \*before\* closing `` tag.

6. Add GSAP CDN: (Crucial Step!)

- o Go to Google and search for "GSAP CDN".
- o Copy the script tag from the official GreenSock website (usually the first result).
- o **Paste it in `index.html` *just above* your `script.js` file link.**

```
<!-- GSAP CDN Link (paste here) -->
<script src="script.js"></script>
```

7. Example Element (in `index.html`):

```
<div id="box"></div>
```

8. Basic CSS (`style.css`):

```
#box {
  height: 300px;
  width: 300px;
  background-color: crimson;
}
body {
  background-color: #333; /* For better visibility */
}
```

GSAP Core Methods: `.to()` and `.from()`

When using GSAP, you typically write `gsap.` followed by a method.

```
gsap.method(target, properties);
```

There are two primary methods for basic animations:

`gsap.to()`: Animate from *Initial* to *Final* Position

- Moves an element from its current (initial) state to a specified (final) state.
- **Analogy:** "Go to" a specific point.
- **Syntax:**

```
gsap.to("#box", {
  x: 1000,
  duration: 2,
  delay: 1
});
```

- **Explanation:**

Initial Position  Moves to 

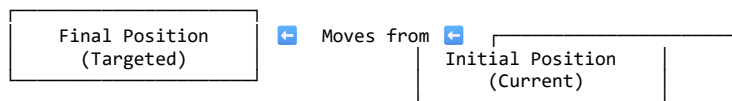


← `gsap.from()`: Animate from *Final* to *Initial* Position

- Moves an element from a specified (final) state **back** to its current (initial) state.
- Useful when you want elements to animate **into** their default HTML/CSS positions.
- **Analogy:** "Come from" a specific point to where it belongs.
- **Syntax:**

```
gsap.from("#box", {
  x: 1200,
  duration: 2,
  delay: 1
});
```

- **Explanation:**



⚙️ Common GSAP Animation Properties

Properties are given inside curly braces `{}` as key-value pairs (like CSS).

- `x`, `y`: Move element along the X or Y axis (e.g., `x: 1000` for 1000 pixels).
- `duration`: **Total time** the animation takes to complete (in seconds, e.g., `duration: 2`).
- `delay`: **Wait time** before the animation starts (in seconds, e.g., `delay: 1`).
- `rotate`: Rotates the element (in degrees, e.g., `rotate: 360`). No `deg` unit needed.
- `scale`: Changes the size of the element (e.g., `scale: 2` for double, `scale: 0.5` for half).
- `opacity`: Controls transparency (e.g., `opacity: 0` for fully transparent, `opacity: 1` for fully opaque).

🎨 Using CSS Properties

You can use most CSS properties directly in GSAP with a crucial rule:

- **Camel Case:** For CSS properties with hyphens (e.g., `background-color`, `border-radius`), convert them to **camelCase**.
 - `background-color` becomes `backgroundColor`.
 - `border-radius` becomes `borderRadius`.
- **Strings:** Values that are words or contain special characters (like `%`) must be in **double quotes**.
 - `backgroundColor: "blue"`
 - `borderRadius: "50%"` (even though `50` could be a number, `%` makes it a string).

```
gsap.to("#box", {
  backgroundColor: "blue", // CamelCase & String
  borderRadius: "50%",    // CamelCase & String
  rotate: 360             // Number, no quotes or 'deg'
});
```

🎯 Animating Multiple Elements

- You can target multiple elements using standard CSS selectors (e.g., `h1`, `.myClass`).
- **Example:** Applying animation to all `h1` elements:

```
gsap.from("h1", {
  opacity: 0,
  y: 20,
  duration: 1
});
```

🚶 Staggering Animations

The `stagger` property allows you to animate multiple elements sequentially, one after another.

- Add `stagger` inside the properties object.
- **Positive value:** Animates elements in their default order.

```
gsap.from("h1", {
  opacity: 0,
  y: 20,
  duration: 0.5,
  stagger: 0.3 // Each h1 animates 0.3s after the previous one
});
```

- **Negative value:** Animates elements in reverse order.

```
stagger: -0.3 // Starts from the last h1
```

- Element 1: [---Animation---]
Element 2: [---Animation---]
Element 3: [---Animation---]

Repeating Animations: `repeat` and `yoyo`

- `repeat`: Specifies how many times an animation should repeat *after* its initial run.
 - `repeat: 1` -> Animation plays 2 times total (1 original + 1 repeat).
 - `repeat: 2` -> Animation plays 3 times total (1 original + 2 repeats).
 - `repeat: -1` -> **Infinite repeat.** 

```
gsap.to("#box", {
  x: 1200,
  duration: 2,
  repeat: -1 // Loop indefinitely
});
```

- `yoyo`: When `true`, reverses the animation on each repeat, creating a back-and-forth effect.
 - Requires `repeat` to be set.
 - **Analogy:** Like a yo-yo toy going down and coming back up. 

```
gsap.to("#box", {
  x: 1200,
  duration: 2,
  repeat: -1,
  yoyo: true // Go to 1200px, then come back, repeat
});
```

GSAP Timeline: Orchestrating Animations

The Problem with Chained Delays

When animating multiple elements sequentially, calculating `delay` for each subsequent animation becomes complex and error-prone, especially with changing `duration` values.

```
Animation 1: duration + delay = total time
Animation 2: delay = Animation 1's total time
Animation 3: delay = Animation 1's total time + Animation 2's duration
```

This manual calculation is difficult to manage. 

The Solution: `gsap.timeline()`

A timeline allows you to orchestrate multiple animations in a **synchronous** (sequential) fashion, like a train where bogies follow one another automatically. 

- **Benefit:** You don't need to manually calculate `delay` for each animation within a timeline; they run in sequence.
- **How it works:**

```
Timeline: [Animation 1] [Animation 2] [Animation 3] ...
```

- **Syntax:**

1. **Create a Timeline:**

```
const tl = gsap.timeline(); // `tl` is a common variable name
```

2. **Add Animations to the Timeline:** Instead of `gsap.to()` or `gsap.from()`, use `tl.to()` or `tl.from()`.

```
tl.to("#box1", { x: 1500, rotate: 360, duration: 1.5, delay: 1 });
tl.to("#box2", { x: 1500, backgroundColor: "yellow", duration: 1.5 }); // No delay needed!
tl.to("#box3", { x: 1500, scale: 0.5, borderRadius: "50%", duration: 1.5 }); // No delay needed!
```

Practical Example: Animating a Navbar with Timeline

Using `tl.from()` to bring elements into view sequentially.

HTML Structure:

```
<nav>
  <h2>Shreyansh</h2>
  <div class="part2">
    <h4>Work</h4>
    <h4>About</h4>
    <h4>Courses</h4>
    <h4>Blog</h4>
  </div>
</nav>
<h1>Shreyansh Coding School</h1>
```

JavaScript (with Timeline):

```
const tl = gsap.timeline();

// Animate H2 (logo)
tl.from("nav h2", {
  y: -30, // Come from -30px on Y-axis
  opacity: 0,
  duration: 1,
  delay: 0.5 // Initial delay for the first element
});

// Animate H4s (nav links) with stagger
tl.from("nav .part2 h4", {
  y: -20, // Come from -20px on Y-axis
  opacity: 0,
  duration: 0.5,
  stagger: 0.3 // Each h4 appears after 0.3s
});

// Animate H1 (main title)
tl.from("h1", {
  y: 20, // Come from 20px on Y-axis
  opacity: 0,
  scale: 0.2,
  duration: 0.5
});
```

This creates a smooth, coordinated entrance animation for all navbar and main title elements. ✨